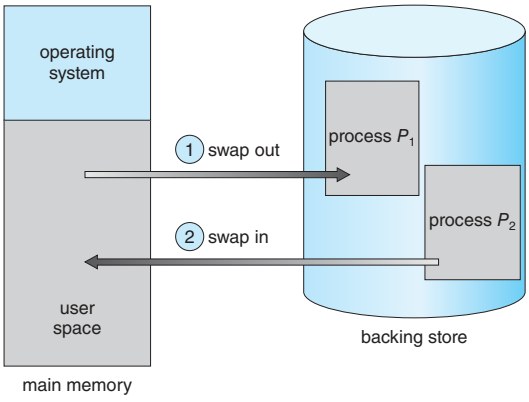
ترجمه‌ی مجازی-به-فیزیکی، زیادی به طول خواهد انجامید در صورتی که هر آدرس مستلزم جستجوی جدول درهم باشد، لذا پردازنده یک TLB پیاده‌سازی می‌کند که وارده‌های جدول ترجمه[1] (TTEهایی) را برای جستجوهای پرسرعت سخت‌افزاری نگه می‌دارد. کَشی از این TTEها در بافر ذخیره‌ی ترجمه[2] (TSB) مقیم است که یک وارده برای هر صفحه‌ی اخیراً دستیابی شده لحاظ می‌کند. موقعی که یک رجوع به آدرس مجازی صورت می‌گیرد، سخت‌افزار، TLB را برای یک ترجمه جستجو می‌کند. اگر چیزی پیدا نشود، سخت‌افزار برای جستجوی TTE متناظر با آدرس مجازی مسبب مراجعه، در TSB داخل حافظه سرکی می‌کشد[3]. این کارکرد **سَرَک TLB** در بسیاری از پردازنده‌های مدرن یافت می‌شود. اگر تطابقی در TSB پیدا شود، پردازنده وارده‌ی TSB را به TLB کپی کرده و ترجمه‌ی حافظه کامل می‌شود. اگر هیچ تطبیقی در TSB پیدا نشود، هسته وقفه می‌گیرد تا جدول درهم را جستجو کند. سپس، هسته یک TTE از جدول درهم مناسبی ایجاد کرده و آن را در TSB برای بارگذاری خودکار به داخل TLB توسط واحد مدیریت حافظه‌ی پردازنده ذخیره می‌نماید. در نهایت، راه‌انداز وقفه، کنترل را به MMU برمی‌گرداند که ترجمه‌ی آدرس را تکمیل کرده و بایت یا کلمه‌ی درخواستی را از حافظه‌ی اصلی بازیابی می‌کند.

۹-۵ مبادله

دستورالعمل‌های پردازه و داده‌هایی که بر روی آن‌ها عملیات دارند بایستی برای اجرا در حافظه قرار بگیرند. با این وجود، یک پردازه یا بخشی از آن می‌تواند موقتاً به بیرون حافظه به **ذخیره‌ی پشتیبان**[4] **مبادله شود**[5] و در ادامه، برای تداوم اجرا به داخل حافظه آورده شود (شکل ۹-۱۹). مبادله، این امکان را به وجود می‌آورد که فضای آدرس فیزیکی تمامی پردازه‌ها در مجموع از حافظه‌ی فیزیکی واقعی سیستم فراتر رود، از این رو درجه‌ی چندبرنامگی در یک سیستم بالا می‌رود.



شکل ۹-۱۹ مبادله‌ی استاندارد دو پردازه با استفاده از یک دیسک به عنوان ذخیره‌ی پشتیبان.

1. Translation table entries
2. Translation storage entry
3. Walks through
4. Backing store
5. Swapped

۹-۵-۱ مبادله‌ی استاندارد

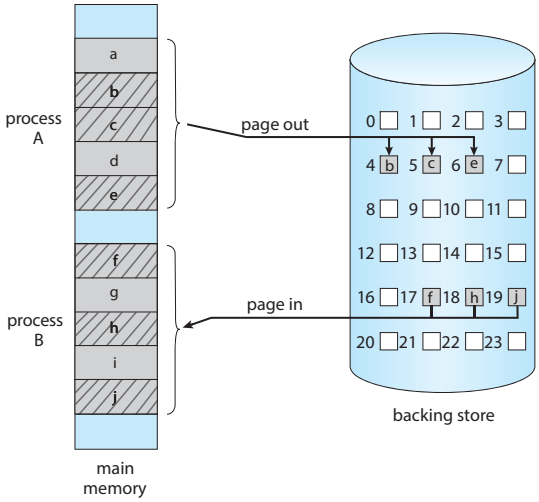
مبادله‌ی استاندارد مشمول جابجایی پردازه‌ها میان حافظه‌ی اصلی و ذخیره‌ی پشتیبان است. ذخیره‌ی پشتیبان در بیشتر موارد یک حافظه‌ی ثانویه‌ی پرسرعت است. بایستی به‌اندازه‌ی کافی بزرگ باشد تا هر بخشی از پردازه‌ها را که نیاز به ذخیره و بازیابی داشته باشند تحت پوشش قرار بدهد و می‌بایست امکان دسترسی مستقیم به این تصاویر حافظه را فراهم نماید. موقعی که یک پردازه یا بخشی از آن به ذخیره‌ی پشیبان مبادله می‌شود، بایستی ساختمان‌داده‌های مرتبط با پردازه در ذخیره‌ی پشتیبان نوشته شوند. برای یک پردازه‌ی چندنخی، تمامی ساختمان‌داده‌های مربوط به هر یک از نخ‌ها نیز بایستی مبادله شوند. سیستم‌عامل می‌بایست برای پردازه‌هایی که مبادله شده‌اند ابرداده‌هایی نیز نگه بدارد تا موقعی که دوباره به حافظه مبادله می‌شوند بتوانند بازیابی شوند.

مزیت مبادله‌ی استاندارد این است که اجازه می‌دهد تا حافظه‌ی فیزیکی بیش از اندازه‌اش مورد استفاده قرار بگیرد، لذا سیستم می‌تواند پردازه‌های بیشتری به نسبت حافظه‌ی فیزیکی واقعی برای ذخیره‌سازی آن‌ها پوشش بدهد. پردازه‌های بیکار یا اکثراً بیکار نامزد خوبی برای مبادله هستند؛ هرگونه حافظه‌ای که به این پردازه‌های غیرفعال تخصیص پیدا کرده بود، می‌تواند به پردازه‌های فعال تخصیص یابد. اگر یک پردازه‌ی غیرفعال که به بیرون مبادله شده است دوباره فعال شود، بایستی دوباره به داخل حافظه مبادله شود. این کار در شکل ۹-۱۹ نمایش داده می‌شود.

۹-۵-۲ مبادله با صفحه‌بندی

مبادله‌ی استاندارد در سیستم‌های یونیکس سابق استفاده می‌شد، اما عموماً دیگر در سیستم‌عامل‌های فعلی استفاده نمی‌شود، چون میزان زمان لازم برای انتقال کامل پردازه‌ها میان حافظه و ذخیره‌ی پشتیبان خیلی بالاست. (یک استثناء در این مورد سولاریس است که هنوز هم از مبادله‌ی استاندارد استفاده می‌کند، با این وجود، فقط تحت شرایط بسیار حادی که حافظه فوق‌العاده پایین است استفاده می‌شود.)

اکثر سیستم‌ها، اعم از لینوکس و ویندوز، هم‌اکنون از گونه‌ای از مبادله که در آن، صفحاتی از یک پردازه—به‌جای پردازه‌ی کامل—می‌توانند مبادله شوند، استفاده می‌کنند. این راهبرد هنوز هم اجازه می‌دهد تا حافظه‌ی فیزیکی، فراتر از میزان آن مورد استفاده قرار بگیرد، اما هزینه‌ی مبادله‌ی کامل پردازه‌ها را ندارد، چرا که تنها تعداد کمی از صفحات مشمول مبادله می‌شوند. در حقیقت، تعبیر *مبادله* هم‌اکنون عموماً به مبادله‌ی استاندارد مربوط می‌شود، و *صفحه‌بندی* به مبادله با صفحه‌بندی مربوط می‌شود. یک عمل **page out** یک صفحه را از حافظه به ذخیره‌ی پشتیبان جابجا می‌کند؛ فرایند معکوس، تحت عنوان **page in** شناخته شده است. مبادله با صفحه‌بندی در شکل ۹-۲۰ نمایش داده می‌شود که زیرمجموعه‌ای از صفحات پردازه‌های A و B به‌ترتیب به بیرون و داخل صفحه‌بندی می‌شوند. همان‌گونه که در فصل ۱۰ خواهیم دید، مبادله با صفحه‌بندی، همراه با حافظه‌ی مجازی بهتر کار می‌کند.



شکل ۹-۲۰ مبادله با صفحه‌بندی.

۹-۵-۳ مبادله در سیستم‌های موبایل

اکثر سیستم‌عامل‌های کامپیوترهای شخصی و کارگزارها مبادله‌ی صفحات را پشتیبانی می‌کنند. در مقابل، سیستم‌های موبایل نوعاً از هیچ یک از اَشکال مبادله پشتیبانی نمی‌کنند. دستگاه‌های موبایل برای ذخیره‌سازی دایمی خود عموماً به جای دیسک‌های سخت که جاگیرتر هستند از حافظه‌ی فلش استفاده می‌کنند. محدودیت فضای حاصل، یکی از دلایلی است که چرا طراحان سیستم‌عامل موبایل از مبادله اجتناب می‌کنند. دلایل دیگر عبارتند از: تعداد محدود نوشتن‌هایی که حافظه‌ی فلش می‌تواند پیش از این که از رده خارج شود تحمل کند و بازده ضعیف میان حافظه‌ی اصلی و حافظه‌ی فلش این دستگاه‌ها.

موقعی که حافظه‌ی آزاد از یک آستانه‌ی معین پایین‌تر می‌رود، iOS اَپل به جای استفاده از مبادله از برنامه‌ها *می‌خواهد* که به صورت داوطلبانه حافظه‌ی تخصیصی خود را رها کنند. داده‌های فقط‌خواندنی (همچون کد) از سیستم حذف شده و بعداً در صورت نیاز از حافظه‌ی فلش بارگذاری می‌شوند. داده‌هایی هم که تغییر یافته‌اند (از قبیل پشته) هرگز حذف نمی‌شوند. با این وجود، هر برنامه‌ای که مبادرت به آزادسازی حافظه‌ی کافی ننماید، ممکن است توسط سیستم‌عامل خاتمه داده شود.

سیستم‌عامل اندروید از مبادله پشتیبانی نکرده و از یک راهبرد مشابه آنچه که در iOS استفاده می‌شود، تبعیت می‌کند. ممکن است در صورت کافی نبودن حافظه‌ی آزاد، یک پردازه را خاتمه بدهد. با این وجود، پیش از خاتمه‌دادن پردازه، **حالت برنامه‌ی** آن را در حافظه‌ی فلش می‌نویسد تا این که بتواند دومرتبه به سرعت شروع به کار کند.

به خاطر این محدودیت‌ها، برنامه‌نویسان سیستم‌های موبایل بایستی در تخصیص و رهایی حافظه دقت کنند تا مطمئن شوند که برنامه‌های آنها از حافظه‌ی خیلی زیادی استفاده نکرده یا گرفتار چکه‌های حافظه نخواهند شد.

کارایی سیستم تحت مبادله

هرچند که مبادله‌ی صفحات مؤثرتر از مبادله‌ی پردازه‌های کامل است، موقعی که یک سیستم از هر شکلی از مبادله استفاده می‌کند، غالباً علامت آن است که پردازه‌های فعال‌تری از میزان حافظه‌ی فیزیکی موجود وجود دارد. عموماً دو رویکرد برای رسیدگی به این وضعیت وجود دارد: (۱) خاتمه‌دادن برخی از پردازه‌ها، یا (۲) خرید حافظه‌ی فیزیکی بیشتر!

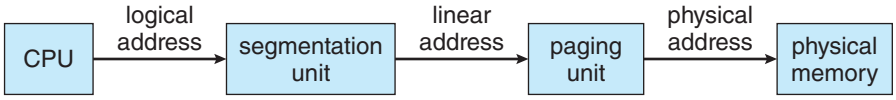
۹-۶ مثال: معماری‌های ۳۲ و ۶۴ بیتی اینتل

معماری تراشه‌های اینتل دهه‌هاست که چشم‌انداز کامپیوتر شخصی را تحت سلطه‌ی خود دارد. اینتل ۸۰۸۶، ۱۶ بیتی در اواخر دهه‌ی ۱۹۷۰ ظهور کرد و خیلی زود با یک تراشه‌ی ۱۶ بیتی دیگر–اینتل ۸۰۸۸–دنبال شد که چون در IBM PC اولیه استفاده می‌شد مورد توجه واقع شد. اینتل بعداً سری تراشه‌های ۳۲ بیتی–IA-32–را تولید کرد که خانواده‌ی پردازنده‌های پنتیوم ۳۲ بیتی را در بر داشت. معماری IA-32 از هر دوی صفحه‌بندی و قطعه‌بندی پشتیبانی می‌کند. در همین اواخر، اینتل سری تراشه‌های ۶۴ بیتی را تولید کرده است که مبتنی بر معماری x86-64 می‌باشند. در حال حاضر، تمامی سیستم‌عامل‌های رایج، اعم از ویندوز، macOS و لینوکس (البته، هرچند که لینوکس بر روی چندین معماری دیگر نیز اجرا می‌شود) بر روی تراشه‌های اینتل اجرا می‌شوند. با این وجود، قابل توجه است که سلطه‌ی اینتل بر سیستم‌های موبایل گسترش پیدا نکرده است، جایی که فعلاً معماری ARM موفقیت قابل توجهی را تجربه می‌کند (بخش ۹-۷ را ببینید).

در این بخش، ترجمه‌ی آدرس را در هر دوی معماری‌های IA-32 و x86-64 بررسی می‌کنیم. با این همه، قبل از پرداختن به این موضوع مهم است که توجه داشته باشید چون اینتل به مرور سال‌ها نسخه‌های متعددی–در کنار گونه‌های آنها–از معماری‌های خود را عرضه کرده است، نمی‌توانیم توصیف کاملی از ساختار مدیریت حافظه‌ی تمامی این تراشه‌ها را ارائه بدهیم. چون نمی‌توانیم تمامی جزئیات پردازنده را فراهم کنیم، بهترین کار این است که این اطلاعات را برای کتاب‌های معماری کامپیوتر بسپاریم. در عوض، اینجا مفاهیم اصلی مدیریت حافظه‌ی پردازنده‌های اینتل را مطرح می‌کنیم.

۹-۶-۱ معماری IA-32

مدیریت حافظه در سیستم‌های IA-32 به دو مؤلفه–قطعه‌بندی و صفحه‌بندی–تقسیم شده و به این صورت کار می‌کند: پردازنده آدرس‌های منطقی را تولید می‌کند که به واحد قطعه‌بندی عرضه می‌شوند. واحد قطعه‌بندی برای هر یک از آدرس‌های منطقی، یک آدرس خطی تولید می‌نماید. سپس، آدرس خطی در اختیار واحد صفحه‌بندی قرار می‌گیرد که آن هم به نوبت خود آدرس فیزیکی حافظه‌ی اصلی را تولید می‌کند. بنابراین، واحدهای قطعه‌بندی و صفحه‌بندی، در کنار هم، واحد مدیریت حافظه (MMU) محسوب می‌شوند. این طرح در شکل ۹-۲۱ نشان داده می‌شود.



شکل ۹-۲۱ ترجمه‌ی آدرس منطقی به فیزیکی در IA-32.

۹-۶-۱-۱ قطعه‌بندی IA-32

معماری IA-32 به یک قطعه اجازه می‌دهد تا ۴ گیگابایت بزرگی داشته باشد و حداکثر تعداد قطعات هر پردازه ۱۶کیلو است. فضای آدرس منطقی هر پردازه به دو پارتیشن تقسیم می‌شود. پارتیشن اول که می‌تواند تا ۸ کیلو قطعه داشته باشد خصوصی پردازه محسوب می‌شود. پارتیشن دوم که می‌تواند تا ۸ کیلو قطعه داشته باشد مابین تمامی پردازه‌ها مشترک است. اطلاعات مربوط به پارتیشن اول در **جدول توصیفگر محلی**[1] (**LDT**) و اطلاعات مربوط به پارتیشن دوم در **جدول توصیفگر سراسری**[2] (**GDT**) نگه‌داری می‌شوند. هر یک از وارده‌های LDT و GDT شامل یک توصیفگر قطعه‌ی ۸ بایتی به همراه اطلاعات جزئی‌تر مربوط به یک قطعه‌ی خاص، اعم از مکان شروع و حد قطعه می‌باشند.

آدرس منطقی به صورت یک زوج (selector, offset) می‌باشد که selector یک عدد ۱۶ بیتی است:

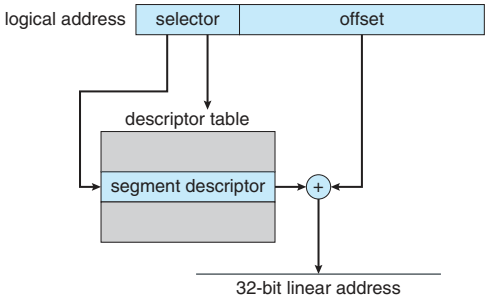
s	g	P
۱۳	۱	۲

در اینجا، s معرف شماره‌ی قطعه، g نشانگر آن است که قطعه در GDT و یا LDT قرار دارد و p به حفاظت مربوط می‌شود. افست، یک عدد ۳۲ بیتی است که محل بایت را در قطعه‌ی مورد نظر مشخص می‌کند.

ماشین دارای شش ثبات قطعه می‌باشد که امکان آدرس‌دهی شش قطعه را در هر لحظه برای پردازه فراهم می‌کنند. همین‌طور، شش ثبات ۸ بایتی با قابلیت ریزبرنامه‌نویسی هم دارد که توصیفگرهای متناظر را از LDT و یا GDT نگه می‌دارند. این امکان، کَش پنتیوم را قادر می‌سازد تا مجبور نشود برای هر بار مراجعه به حافظه، توصیفگر را از حافظه بخواند.

آدرس خطی IA-32، ۳۲ بیت طول دارد و به این صورت شکل می‌گیرد. ثبات قطعه به وارده‌ی مناسبی از LDT یا GDT اشاره می‌کند. با استفاده از اطلاعات مبنا و حد مربوط به قطعه‌ی مورد نظر، یک **آدرس خطی** تولید می‌شود. در ابتدا، اعتبار آدرس با استفاده از حد بررسی می‌شود. اگر آدرس معتبر نباشد، یک خطای حافظه تولید می‌شود که حاصل آن، تله‌ای به سیستم‌عامل خواهد بود. اگر معتبر باشد، در این صورت مقدار افست به مقدار مبنا افزوده می‌شود تا یک آدرس خطی ۳۲ بیتی بدست آید. این روال در شکل ۹-۲۲ نشان داده می‌شود. در بخش بعدی، چگونگی تبدیل این آدرس خطی به یک آدرس فیزیکی را توسط واحد صفحه‌بندی بحث می‌کنیم.

1. Local descriptor table
2. Global descriptor table



شکل ۹–۲۲ قطعه‌بندی IA-32.

۹–۶–۱–۲ صفحه‌بندی IA-32

معماری IA-32 داشتن صفحاتی با اندازه‌ی ۴ کیلوبایتی و یا ۴ مگابایتی را ممکن می‌سازد. IA-32 برای صفحات ۴ کیلوبایتی از یک طرح صفحه‌بندی دوسطحی استفاده می‌کند که تقسیم‌بندی آدرس خطی ۳۲ بیتی آن به صورت زیر است:

شماره‌ی صفحه		افست صفحه
p_1	p_2	d
۱۰	۱۰	۱۲

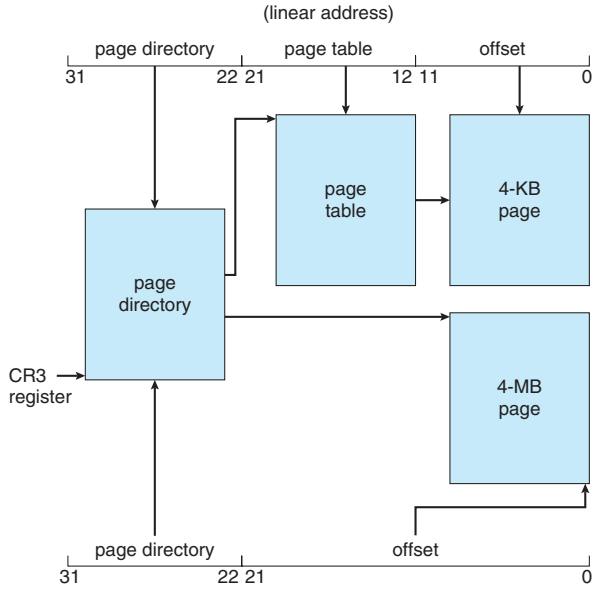
طرح ترجمه‌ی آدرس این معماری مشابه طرحی است که در شکل ۹–۱۶ نشان داده می‌شود. جزئیات بیشتر مربوط به ترجمه‌ی آدرس پنتیوم اینتل در ۹–۲۳ نشان داده می‌شود. ۱۰ بیت با ارزش به بیرونی‌ترین جدول صفحه رجوع دارند که تعبیر IA-32 از آن، **دایرکتوری صفحه** است. (ثبات CR3 به دایرکتوری صفحه‌ی پردازه‌ی جاری اشاره می‌کند.) وارده‌ی دایرکتوری صفحه به یک جدول صفحه‌ی داخلی اشاره می‌کند که با محتوای ۱۰ بیت میانی آدرس خطی ایندکس می‌شود. سرانجام، بیت‌های کم ارزش ۰ تا ۱۱ به افست داخل صفحه‌ی ۴ کیلوبایتی که جدول صفحه اشاره می‌کند مربوط می‌شوند.

پرچم Page_Size یک وارده در دایرکتوری صفحه است که—در صورت انتخاب— نشان می‌دهد اندازه‌ی قاب‌صفحه، ۴ مگابایتی است و ۴ کیلوبایت معمول نیست. در صورت انتخاب این پرچم، دایرکتوری صفحه با دورزدن جدول صفحه‌ی داخلی، مستقیماً به یک قاب صفحه‌ی ۴ مگابایتی اشاره می‌کند و ۲۲ بیت کم ارزش آدرس خطی، افست قاب‌صفحه‌ی ۴ مگابایتی می‌باشند.

جهت بهبود کارآمدی استفاده از حافظه‌ی فیزیکی می‌توان جدول صفحات IA-32 را به دیسک مبادله کرد. در این حالت، از یک بیت نامعتبری در وارده‌ی دایرکتوری صفحه استفاده می‌شود که نشان می‌دهد آیا جدولی که وارده به آن اشاره دارد در حافظه قرار دارد و یا در دیسک است. اگر جدول روی دیسک باشد، سیستم‌عامل با استفاده از ۳۱ بیت باقیمانده می‌تواند مکان جدول را در دیسک مشخص کند. سپس، جدول در صورت نیاز می‌تواند به داخل حافظه آورده شود.

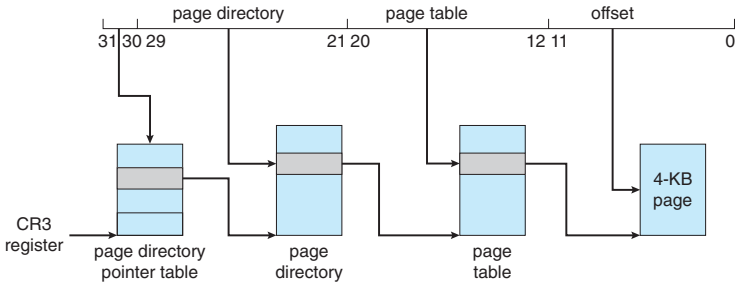
همچنان که توسعه‌دهندگان نرم‌افزار شروع کردند تا محدودیت‌های حافظه‌ی ۴ گیگابایت معماری‌های ۳۲ بیتی را بردارند، اینتل یک **گسترش آدرس صفحه**[1] (**PAE**) عرضه کرد که برای پردازنده‌های ۳۲ بیتی، امکان دسترسی فضای آدرس فیزیکی بزرگ‌تر از ۴ گیگابایت را فراهم می‌کند. تفاوت اساسی که با پشتیبانی PAE مطرح شد این بود که صفحه‌بندی از یک طرح دوسطحی (چنان که در شکل ۹-۲۳ نشان داده می‌شود) به یک طرح سه‌سطحی قدم گذاشت که در آن، دو بیت بالایی به یک **جدول اشاره‌گر دایرکتوری صفحه** رجوع می‌کنند. شکل ۹-۲۴، سیستم PAE را با صفحات ۴ کیلوبایتی نشان می‌دهد. (PAE از صفحات ۲ مگابایتی هم پشتیبانی می‌کند.)

PAE، اندازه‌ی واردهای جدول صفحه و دایرکتوری صفحه را هم از ۳۲ به ۶۴ بیت افزایش داد که به آدرس مبنای جدول صفحات و قاب صفحات امکان می‌دهد از ۲۰ به ۲۴ بیت گسترش داشته باشند. در ترکیب با افست ۱۲ بیتی، افزودن پشتیبانی PAE به IA-32، فضای آدرس را به ۳۶ بیت افزایش داد که تا ۶۴ گیگابایت از حافظه‌ی فیزیکی را پشتیبانی می‌کند. توجه به این نکته مهم است که پشتیبانی سیستم‌عامل برای استفاده از PAE لازم است. هر دوی لینوکس و macOS از PAE پشتیبانی می‌کنند. با این وجود، نسخه‌های ۳۲ بیتی سیستم‌عامل‌های ویندوز رومیزی همچنان فقط از ۴ گیگابایت از حافظه‌ی فیزیکی، حتی با فعال بودن PAE پشتیبانی می‌کنند.



شکل ۹-۲۳ صفحه‌بندی معماری IA-32.

1. Page address extension

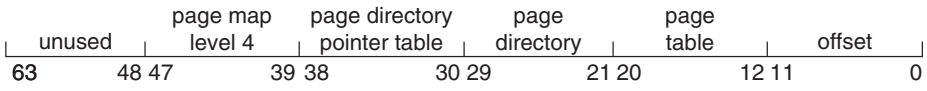


شکل ۹–۲۴ گسترش‌های آدرس صفحه.

۹–۶–۲ x86-64

اینتل در توسعه‌ی معماری‌های ۶۴ بیتی تاریخچه‌ی جالبی داشته است. ورودی آغازین آن IA-64 (بعداً **Itanium** نام گرفت) بود، اما این معماری زیاد فراگیر نشد. در این میان، سازنده‌ی دیگر تراشه–AMD–توسعه‌ی یک معماری ۶۴ بیتی را تحت عنوان x86-64 شروع کرد که مبتنی بر توسعه‌ی مجموعه دستورالعمل IA-32 موجود بود. x86-64 از فضاهای آدرس منطقی و فیزیکی بسیار بزرگ‌تر، در کنار چندین پیشرفت معماری دیگر پشتیبانی می‌کرد. از نظر تاریخی، AMD غالباً تراشه‌هایش را مطابق معماری اینتل توسعه داده بود، اما اکنون نقش‌ها برعکس شده است چرا که اینتل از معماری x86-64 معماری AMD تبعیت می‌کند. در مباحث این معماری، به جای استفاده از اسامی معماری **AMD64** و **اینتل ۶۴**، از اصطلاح کلی‌تر **x86-64** استفاده خواهیم کرد.

پشتیبانی فضای آدرس ۶۴ بیتی به یک حافظه‌ی قابل آدرس‌دهی باورنکردنی 2^{64} بایت منجر می‌شود–عددی بزرگ‌تر از ۱۶ کوینتیلیون[1] (یا ۱۶ اگزا بایت). با این وجود، هرچند که سیستم‌های ۶۴ بیتی می‌توانند به صورت بالقوه حافظه‌ی به این بزرگی را آدرس‌دهی کنند، در عمل به مراتب کمتر از ۶۴ بیت برای نمایش آدرس در طراحی‌های فعلی استفاده می‌شود. معماری x86-64 فعلاً یک فضای آدرس مجازی ۶۴ بیتی با پشتیبانی اندازه‌ی صفحات ۴ کیلوبایت، ۲ مگابایت یا ۱ گیگابایت را با استفاده از چهار سطح سلسله‌مراتب صفحه‌بندی فراهم می‌کند. نمایش آدرس خطی در شکل ۹–۲۵ نشان داده می‌شود. چون این طرح آدرس‌دهی می‌تواند از PAE استفاده کند، آدرس‌های مجازی در اندازه‌ی ۴۸ بیت هستند اما از آدرس‌های فیزیکی ۵۲ بیتی (۴۰۹۶ ترابایت) پشتیبانی می‌کنند.



شکل ۹–۲۵ آدرس خطی x86-64.

1. Quintillion

محاسبات ۶۴ بیتی

تاریخ به ما آموخته است که هرچند ظرفیت‌های حافظه، سرعت‌های پردازنده و قابلیت‌های کامپیوتری مشابه ظاهراً به قدر کافی بزرگ هستند تا نیازمان را برای آینده‌ی قابل پیش‌بینی ارضا نمایند، سرانجام رشد فناوری، ظرفیت‌های موجود را جذب می‌کند و ما خود را نیازمند حافظه یا قدرت پردازشی اضافی می‌بینیم، غالباً زودتر از آنچه که تصور می‌کردیم. ممکن است آینده‌ی فناوری چه چیزی به ارمغان آورد که فضای آدرس ۶۴ بیتی را هم بسیار کوچک ببیند؟

۹-۷ مثال: معماری ARMv8

اگرچه بیش از ۳۰ سال است که تراشه‌های اینتل بر بازار کامپیوتر سلطه پیدا کرده‌اند، در عوض، تراشه‌های دستگاه‌های موبایل از قبیل گوشی‌های هوشمند و کامپیوترهای تبلت غالباً روی پردازنده‌های ARM اجرا می‌شوند. جالب‌تر این که، در حالی که اینتل هر دو کار طراحی و ساخت تراشه‌ها را انجام می‌دهد، ARM فقط آنها را طراحی می‌کند. سپس، مجوز طراحی‌های معماری خود را در اختیار سازندگان تراشه می‌گذارد. اَپل مجوز طراحی ARM را برای دستگاه‌های موبایل iPhone و iPad گرفته است و اکثر گوشی‌های هوشمند مبتنی بر اندروید نیز از پردازنده‌های ARM استفاده می‌کنند. ARM علاوه بر دستگاه‌های موبایل، طراحی‌های معماری سیستم‌های تعبیه‌شده‌ی بلادرنگ را نیز فراهم می‌کند. به‌خاطر فراوانی دستگاه‌هایی که روی معماری ARM اجرا می‌شوند، بیش از ۱۰۰ میلیارد پردازنده‌ی ARM تولید شده است، این معماری موقع سنجش از نظر تعداد تراشه‌های تولید شده، فراگیرترین معماری محسوب می‌شود. در این بخش، معماری ARMv8 ۶۴ بیتی را مطرح می‌کنیم.

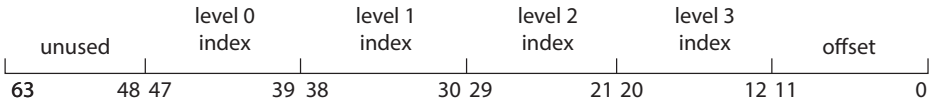
ARMv8 دارای سه **دانه‌ی ترجمه‌ی**[1] متفاوت است: ۴ کیلوبایتی، ۱۶ کیلوبایتی، و ۶۴ کیلوبایتی. هر دانه‌ی ترجمه اندازه‌ی صفحات متفاوتی را فراهم می‌کند، همین‌طور بخش‌های بزرگ‌تری از حافظه‌ی همجوار، تحت عنوان **نواحی**[2]. اندازه‌های صفحه و ناحیه برای دانه‌های ترجمه‌ی متفاوت در زیر نشان داده می‌شوند:

Translation Granule Size	Page Size	Region Size
4 KB	4 KB	2 MB, 1 GB
16 KB	16 KB	32 MB
64 KB	64 KB	512 MB

در دانه‌های ۴ کیلوبایتی و ۱۶ کیلوبایتی می‌توان تا چهار سطح از صفحه‌بندی و برای دانه‌های ۶۴ کیلوبایتی می‌توان تا سه سطح از صفحه‌بندی را استفاده کرد. شکل ۹-۲۶، ساختار آدرس ARMv8 را برای دانه‌ی ترجمه‌ی ۴ کیلوبایتی تا چهار سطح از صفحه‌بندی نشان می‌دهد. (توجه داشته باشید که هرچند ARMv8 یک معماری ۶۴ بیتی است، فقط ۴۸ بیت آن در حال حاضر استفاده می‌شود.) ساختار صفحه‌بندی سلسله‌مراتبی چهارسطحی برای دانه‌ی ترجمه‌ی ۴

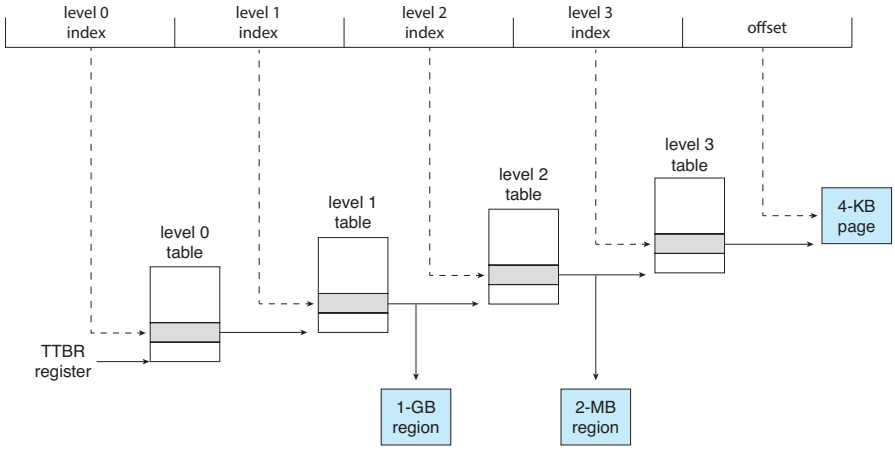
1. Translation granules
2. Regions

کیلوبایتی در شکل ۹–۲۷ نمایش داده می‌شود. (ثبات TTBR، **ثبات مبنای جدول ترجمه** بوده و به جدول سطح ۰ نخ جاری اشاره می‌کند.)



شکل ۹–۲۶ دانه‌ی ترجمه‌ی ۴ کیلوبایتی ARM.

اگر تمامی چهارسطح استفاده شوند، افست (بیت‌های ۱۱–۰ در شکل ۹–۲۶) به افست میان یک صفحه‌ی ۴ کیلوبایتی رجوع می‌کند. با این وجود، توجه داشته باشید که واردهای جدول برای سطح ۱ و سطح ۲ ممکن است به جدول دیگری و یا به یک ناحیه‌ی ۱ گیگابایتی (جدول سطح ۱) و یا ناحیه‌ی ۲ مگابایتی (جدول سطح ۲) رجوع کنند. به‌عنوان یک مثال، اگر جدول سطح ۱ به‌جای یک جدول سطح ۲، به یک ناحیه‌ی ۱ گیگابایتی رجوع کند، ۳۰ بیت کم‌ارزش (بیت‌های ۲۹–۰ در شکل ۹–۲۶) به‌عنوان یک افست به این ناحیه‌ی ۱ گیگابایتی استفاده می‌شوند. با تشابهی، اگر جدول سطح ۲ به‌جای یک جدول سطح ۳، به یک ناحیه‌ی ۲ مگابایتی رجوع کند، ۲۱ بیت کم‌ارزش (بیت‌های ۲۰–۰ در شکل ۹–۲۶) به افست میان این ناحیه‌ی ۲ مگابایتی رجوع می‌کنند.



شکل ۹–۲۷ صفحه‌بندی سلسله‌مراتبی چهارسطحی ARM.

معماری ARM از دو سطح TLB هم پشتیبانی می‌کند. در سطح داخلی، دو **ریز TLB**[1]–یک TLB جداگانه برای داده‌ها و دیگری برای دستورالعمل‌ها–وجود دارند. ریزTLB از ASIDها هم پشتیبانی می‌کند. در سطح بیرونی، یک **TLB اصلی** منفرد وجود دارد. ترجمه‌ی آدرس در سطح ریزTLB شروع می‌شود. در صورت رخداد یک شکست، TLB اصلی بررسی می‌شود. اگر هر دو TLB منجر به شکست شوند، بایستی سرکی به جدول صفحه در سخت‌افزار صورت بگیرد.

1. Micro TLB